SPARKLAB
(HarvestMind)

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team **SparkLab***
Team Repo URL	*https://github.com/mo-omen/HarvestMind/blob/main/README.md*
Project Board URL	*https://github.com/mo-omen/HarvestMind/blob/main/README.md*
Live Deployment URL (optional for preliminary round)	
'Lite' Version URL	*https://sparklab-0yw7.onrender.com/markets.html*

Objective:

The primary objective is to ensure that HarvestMind can reliably analyze real time agricultural data (market prices, weather, and news signals) and generate accurate, actionable recommendations for farmers using AI driven reasoning under varying system conditions.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- **Conversational Farm Intake:** AI chat input
- **AI Decision Orchestrator:** GLM reasoning engine
- **Live Market Monitoring:** FAMA + scraping
- **Weather Data Integration:** Open-Meteo API
- **Signal Cards Generation:** Risk/opportunity insights
- **Frontend Dashboard display:** charts + results

1.2 Out-of-Scope

- User authentication & account management
- Mobile application version
- Advanced personalization (long-term memory)
- App Theme

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Incorrect AI Recommendation	*4*	*5*	*20 (Critical)*	*Cross-reference multiple data sources (weather + price + news)* *Add confidence indicators in UI*	*Provide conflicting inputs and verify consistency*
External API Failure (Weather / Prices)	*3*	*4*	*12 (High)*	*Use fallback cached data* *Graceful UI degradation*	*Simulate API timeout and observe system response*
Slow AI Response (GLM latency)	*3*	*3*	*9 (Medium)*	*Use async processing + Redis queue*	*Measure response time under multiple requests*
Data Scraping Failure (Firecrawl)	*2*	*4*	*15 (High)*	*Backup static datasets*	*Disable scraper and check fallback*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - *Scope: API logic, data parsing, validation*
  - *Execution: PyTest during development + CI pipeline*
  - *Isolation: External APIs mocked*
  - *Pass Condition: Correct outputs for valid + invalid inputs*

- **Integration Test**
  - *Scope: Frontend - Node.js Proxy - FastAPI - APIs*
  - *Execution: After merging services*
  - *Workflow: Real API calls (weather, price, GLM)*
  - *Pass Condition: End-to-end data flow works correctly*

- **Test Environment (CI/CD Practice):**
  - *Local Testing: Docker-based environment*
  - *Staging: Containerized deployment using Docker Compose*
  - *CI/CD: GitHub Actions for automated tests*

- **Regression Testing & Pass/Fail Rules:**
  - ***Execution Phase:*** *Regression testing is conducted across all in-scope features (AI decision engine, data integrations, and frontend dashboard) whenever new code is merged into the main branch to ensure existing functionalities remain unaffected.*

  - ***Pass/Fail Condition:*** *A test case is considered passed only when the actual output fully matches the expected result. Any deviation, including incorrect data, missing UI elements, or performance issues, is marked as a failure and must be recorded in the defect log for resolution.*

  - ***Continuation Rule:*** *Testing progresses in stages, where critical and foundational test cases must meet the defined success threshold before proceeding to subsequent testing phases to ensure system stability.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Test Data Strategy:**
    - ***Manual:*** *Mock datasets are created to simulate real world farming scenarios, including different crop types (e.g., rice, palm oil), varying weather conditions, and diverse market price trends.*
    - ***Automated:*** *Predefined seed scripts and structured datasets are used to generate consistent and repeatable test data for automated testing, ensuring reliability across multiple test runs.*

- **Passing Rate Threshold:**
    - *A minimum of 85% of all executed test cases must pass for the system to be considered stable.*
    - *For critical functionalities including AI decision outputs and external data integrations a 100% pass rate is mandatory, as failures in these areas can significantly impact system reliability and user trust.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section defines the key performance metrics and quality thresholds required for system progression. If any defined threshold is not met, the CI/CD pipeline will block further integration or deployment to ensure system stability and reliability.

4.1 Integration Thresholds (Merging to Main)

The main branch serves as the stable source of truth for the system. Therefore, strict continuous integration checks are enforced before any code is merged.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***5 Errors***	***Failed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***97%***	***Failed***
Test Coverage	***Minimum 81.2%***	***79%***	***Failed***

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***78%***	***Failed***
Critical Bugs	***Zero P0/01 Bugs***	***2***	***Failed***
API Performance	***Response Time < 800ms***	***450ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

This section outlines the defined test cases used to validate system functionality and performance. The test suite includes:

- ***Happy Case (Entire Flow):*** *Validates the complete user journey from input to AI-generated recommendation*
- ***Specific Case (Negative/Edge Test):*** *Ensures the system handles invalid or incomplete inputs gracefully*
- **Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	***Happy Case (Entire Flow):*** *AI Decision Orchestrator*	*Verifies that a user can input farm details, trigger the AI decision engine, and receive a complete, data-driven recommendation with integrated weather, price, and news signals.*	*1. Open the dashboard* *2. Input farm details via conversational interface* *3. Submit query* *4. System retrieves weather, market, and news data* *5. AI processes inputs and generates suggestion*	*AI suggestion is generated successfully* *Output reflects real-time data sources* *Response is displayed within acceptable time (< 5–8 seconds)*	*Recommendation generated successfully* *Data sources integrated correctly* *Response delivered in 6.25 seconds* *Status: Passed*
TC-02	***Specific Case (Negative):*** *Input Validation*	*Ensures the system properly handles incomplete or invalid user input without triggering system errors.*	*1. Open the system* *2. Submit empty or incomplete farm data* *3. Attempt to trigger AI response*	*- System blocks request* *-Clear error message is displayed* *-No backend processing is triggered*	*Request is blocked successfully* *Error message not clearly displayed to user* *Status: Failed*
TC-03	***NFR (Performance):*** *API*	*Evaluates system responsiveness under concurrent usage.*	*1. Send multiple API requests (≈50 concurrent*	*-Average response time < 800ms*	*-Average Response time is received as*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
	Performance		*users)* *2. Measure average response time*	*-0% error rate*	*1128ms* *-0% Error Rate.* *Status: Failed*
TC-04	***NFR (Performance):*** *Dashboard Rendering*	*Evaluates how quickly the frontend dashboard loads and displays data after user interaction.*	*Open the HarvestMind dashboard* *Submit a farm query* *Measure time taken for AI output*	*-Initial page load < 2 seconds* *-Data visualization rendered < 3 seconds after response* *-No UI freezing or lag*	*- Initial load: 2.4 seconds* *- Chart rendering: 3.6 seconds* *- Minor delay observed when loading multiple datasets* ***Status:*** *Failed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

This section validates that the GLM-based AI system produces accurate, relevant, and reliable outputs while handling abnormal or extreme inputs appropriately.

6.1. **Prompt/Response Test Pairs**

3 prompt response pairs are defined to evaluate AI output quality against acceptance criteria.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Should I harvest rice early due to upcoming weather conditions in Kedah?"	- Mentions weather risk - References crop stage - Provides clear recommendation - No fabricated data	Based on recent weather forecasts indicating increased rainfall in Kedah over the next 5–7 days, there is a moderate risk of crop damage if harvesting is delayed. Current market prices for rice remain stable with a slight upward trend. If your crop is near maturity, early harvesting is recommended to minimize potential weather-related losses. However, if the crop is still in early growth stages, continued monitoring is advised.	Pass
AI-02	"Analyze current palm oil market trend in Malaysia"	- Uses market trend signals - Provides reasoning - No hallucinated values	The palm oil market in Malaysia is currently experiencing moderate price stability with slight upward movement due to export demand and supply constraints. Recent reports suggest increased demand from international buyers, while weather conditions have slightly impacted production levels. This combination supports a cautious but positive outlook for short-term pricing.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-03	"What should I do if vegetable prices drop next week?"	- Suggests actionable strategies - References market uncertainty - No unrealistic guarantees	If vegetable prices are expected to decline next week, you may consider harvesting earlier to secure current market rates, especially if your crops are ready. Alternatively, delaying harvest could be beneficial if storage is available and prices are expected to recover.	Pass

6.2. Oversized/Larger Input Test

This test evaluates system behavior when input exceeds defined limits.

Fields	Details
Maximum Input Size	~1500 Tokens
Input used while testing	~9000 Words - 6x of the limit
Expected Behavior	Input is rejected or segmented into smaller chunks
Actual Behavior	- Reject input with error message, OR - Automatically truncate / chunk input
Status	Pass/Fail

6.3. Adversarial/Edge Prompt Test

This test evaluates system robustness against ambiguous or malformed prompts.

- Example prompt:
  "Provide farming advice without using any real data."
- Expected behavior:
  - System either provides a safe, generalized response or warns about lack of reliable data

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.4. Hallucinating Handling

The system mitigates hallucinations by:

- Grounding responses in real-time data sources (weather, prices, news)
- Cross-referencing multiple inputs before generating outputs
- Avoiding unsupported claims when data is unavailable
- Providing transparent reasoning in AI-generated responses

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my